

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра ВТ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Основы машинного обучения»
Тема: «Регрессия»

Студент гр. 2308

Кислицин К. Р.

Преподаватель

Турсуков Н. О.

Санкт-Петербург

2024

Содержание

Цель работы	3
Задачи работы	3
Теоретическая информация.....	3
Ход работы.....	6
Выводы	18
Приложения	19

Цель работы

Целью работы является получение и закрепление навыков предобработки данных и применения методов машинного обучения для решения задач регрессии.

Задачи работы

1. Предварительная обработка данных (аналогично л/р 1)
 - Визуализация значимых признаков (диаграммы рассеяния, ящики с усами, гистограммы)
 - Очистка данных (удаление пропусков, нормализация, удаление дубликатов)
 - Корреляция данных (матрица корреляций)
2. Обучение моделей и подбор параметров:
 - Линейная регрессия
 - LASSO
 - Ридж-регрессия
3. Оценка моделей
 - Вывод метрик
 - Построение графиков

Теоретическая информация

Визуализация значимых признаков

- Диаграммы рассеяния (scatter plots): визуализируют зависимость между двумя количественными признаками, что помогает выявить линейные и нелинейные взаимосвязи, а также обнаружить выбросы.
- Ящики с усами (boxplots): позволяют оценить распределение признаков, медиану, квартильные значения и возможные выбросы в данных.
- Гистограммы (histograms): отображают распределение данных по категориям или интервалам для одного признака, помогают оценить симметрию, наличие выбросов и форму распределения.

Матрица корреляций – квадратная таблица, заголовками строк и столбцов которой являются обрабатываемые переменные, а на пересечении строк и столбцов выводятся коэффициенты корреляции для соответствующей пары признаков.

Линейная регрессия

- Линейная регрессия — это метод статистического анализа, который используется для моделирования зависимостей между одной зависимой переменной и одной или несколькими независимыми переменными. Целью является нахождение линейной функции, которая наилучшим образом аппроксимирует данные.
- Модель строится путём минимизации суммы квадратов ошибок между предсказанными значениями и реальными данными. Это означает, что линейная регрессия находит такие коэффициенты (параметры), при которых линия или плоскость максимально приближены к данным.
- Подбор параметров: Основным параметром — это коэффициенты регрессии, которые вычисляются с помощью методов оптимизации, таких как метод наименьших квадратов. Важно также учитывать метрику ошибки (обычно это среднеквадратичная ошибка) для оценки качества модели.

LASSO (Least Absolute Shrinkage and Selection Operator)

- LASSO — это метод линейной регрессии, который применяется для выбора признаков и регуляризации модели. Он использует L1-норму штрафа, что способствует уменьшению некоторых коэффициентов регрессии до нуля, тем самым исключая соответствующие признаки.
- Модель обучается на данных с целью минимизировать ошибку прогноза, одновременно уменьшая абсолютные значения коэффициентов регрессии. Это помогает избежать переобучения, особенно когда в данных много признаков или признаки имеют высокую корреляцию.

- Подбор параметров: Основной параметр — это коэффициент регуляризации (λ), который контролирует степень штрафа за большие коэффициенты. Чем больше λ , тем сильнее регуляризация. Также важен выбор метрики ошибки (например, среднеквадратичной ошибки) и метода оптимизации.

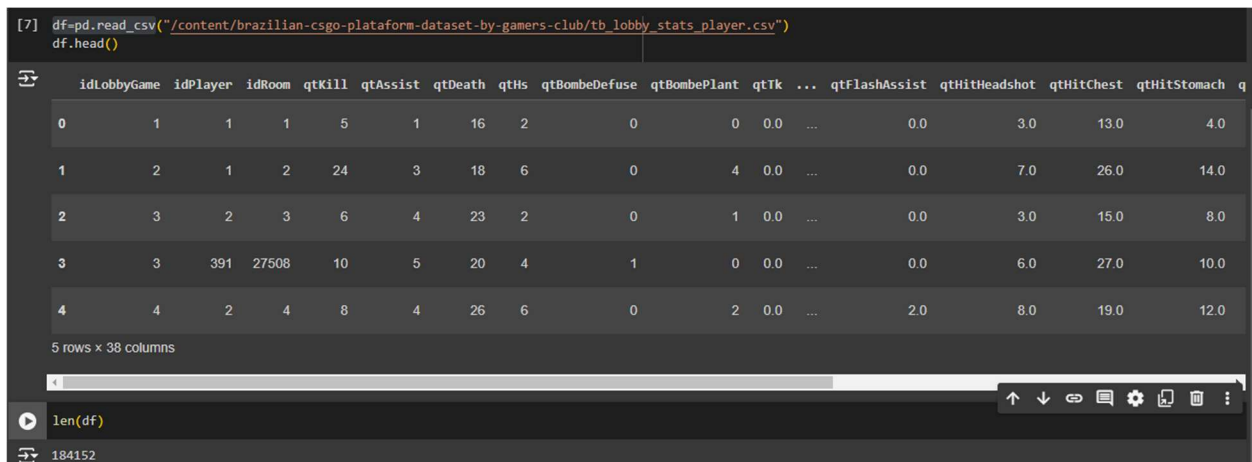
Ридж-регрессия (Ridge Regression)

- Ридж-регрессия — это метод линейной регрессии, который использует L2-норму регуляризации для предотвращения переобучения модели. В отличие от обычной линейной регрессии, которая минимизирует только ошибку предсказания, ридж-регрессия добавляет дополнительный штраф за большие коэффициенты регрессии. Это помогает улучшить обобщающую способность модели, особенно при наличии мультиколлинеарности между признаками.
- Модель обучается путём минимизации суммы квадратов ошибок плюс сумма квадратов коэффициентов, умноженная на коэффициент регуляризации (λ). Это приводит к тому, что модель выбирает более "малые" коэффициенты для признаков, что снижает сложность модели и уменьшает её склонность к переобучению.
- Подбор параметров: Основной параметр — это коэффициент регуляризации (λ), который контролирует степень штрафа за большие коэффициенты. Чем больше λ , тем сильнее регуляризация. Также важен выбор метрики ошибки (например, среднеквадратичной ошибки) для оценки качества модели.

Ход работы

В рамках данной лабораторной работы был рассмотрен датасет по статистике игроков бразильского геймерского клуба в CS:GO. Датасет содержит в себе 184152 строки и 38 признаков.

Сформировал датафрейм на основе загруженных данных и вывел его первые 5 значений (Рисунок 1).



```
[7]: df=pd.read_csv("../content/brazilian-csgo-plataform-dataset-by-gamers-club/tb_lobby_stats_player.csv")
df.head()
```

	idLobbyGame	idPlayer	idRoom	qtKill	qtAssist	qtDeath	qtHs	qtBombeDefuse	qtBombePlant	qtTk	...	qtFlashAssist	qtHitHeadshot	qtHitChest	qtHitStomach	q
0	1	1	1	5	1	16	2	0	0	0.0	...	0.0	3.0	13.0	4.0	
1	2	1	2	24	3	18	6	0	4	0.0	...	0.0	7.0	26.0	14.0	
2	3	2	3	6	4	23	2	0	1	0.0	...	0.0	3.0	15.0	8.0	
3	3	391	27508	10	5	20	4	1	0	0.0	...	0.0	6.0	27.0	10.0	
4	4	2	4	8	4	26	6	0	2	0.0	...	2.0	8.0	19.0	12.0	

5 rows x 38 columns

```
len(df)
```

184152

Рисунок 1 – Предосмотр датафрейма

Построил карту пропусков, пропусков не оказалось, потому добавил их искусственно в два столбца. Соответствующие действия показаны на Рисунке 2.

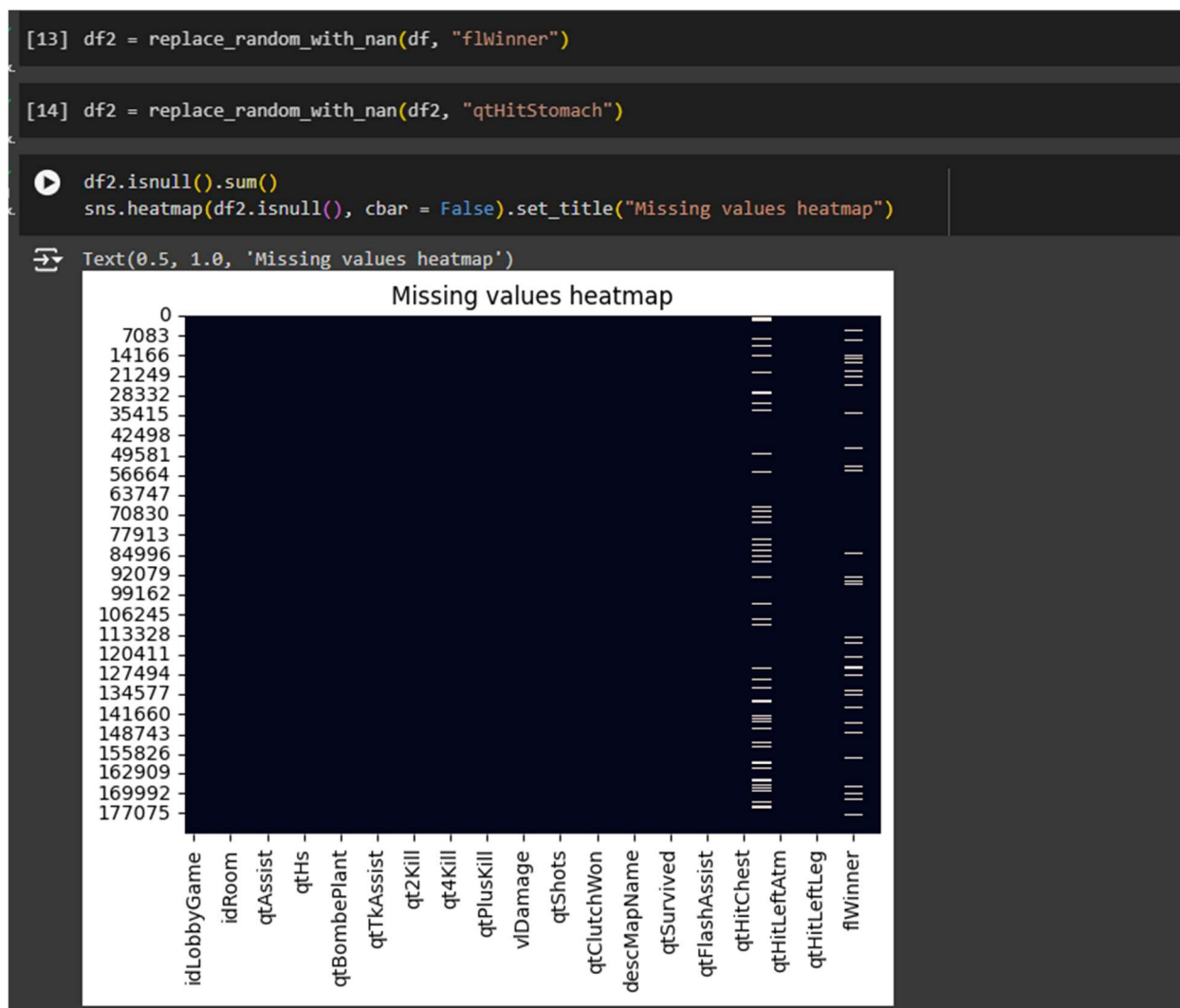


Рисунок 2 – Добавление пропусков

Посмотрел тип данных в столбцах для того, чтобы определить дальнейшие действия по обработке. Нашлось два признака с типом данных «object»: название карты и дата создания записи. Дату разбил на несколько столбцов, название – закодировал. Убрал строки, в которых есть пустые значения для предсказываемого параметра. Все действия приведены на рисунках 3-4.

```
df2.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 184152 entries, 0 to 184151
Data columns (total 38 columns):
 #   Column              Non-Null Count  Dtype  
---  --
 0   idLobbyGame         184152 non-null  int64  
 1   idPlayer            184152 non-null  int64  
 2   idRoom              184152 non-null  int64  
 3   qtKill              184152 non-null  int64  
 4   qtAssist            184152 non-null  int64  
 5   qtDeath             184152 non-null  int64  
 6   qtHs               184152 non-null  int64  
 7   qtBombeDefuse       184152 non-null  int64  
 8   qtBombePlant        184152 non-null  int64  
 9   qtTk               184032 non-null  float64 
10  qtTkAssist          184032 non-null  float64 
11  qt1Kill             184152 non-null  int64  
12  qt2Kill             184152 non-null  int64  
13  qt3Kill             184152 non-null  int64  
14  qt4Kill             184152 non-null  int64  
15  qt5Kill             184152 non-null  int64  
16  qtPlusKill          184152 non-null  int64  
17  qtFirstKill         184152 non-null  int64  
18  vlDamage            184152 non-null  int64  
19  qtHits              184032 non-null  float64 
20  qtShots             184152 non-null  int64  
21  qtLastAlive         184032 non-null  float64 
22  qtClutchWon         184152 non-null  int64  
23  qtRoundsPlayed      184152 non-null  int64  
24  descMapName         184152 non-null  object  
25  vlLevel            184152 non-null  int64  
26  qtSurvived          183447 non-null  float64 
27  qtTrade             183447 non-null  float64 
28  qtFlashAssist       183447 non-null  float64 
29  qtHitHeadshot       183447 non-null  float64 
30  qtHitChest          183447 non-null  float64 
31  qtHitStomach        165089 non-null  float64 
32  qtHitLeftArm        183447 non-null  float64 
33  qtHitRightArm       183447 non-null  float64 
34  qtHitLeftLeg        183447 non-null  float64 
35  qtHitRightLeg       183447 non-null  float64 
36  flWinner            165737 non-null  float64 
37  dtCreatedAt         184152 non-null  object  
dtypes: float64(15), int64(21), object(2)
memory usage: 53.4+ MB
```

Рисунок 3 – Вывод типов данных

```
[18] df2 = df2.dropna(subset=['flWinner'])

[21] df2['dtCreatedAt'] = pd.to_datetime(df2['dtCreatedAt'])

df2['year'] = df2['dtCreatedAt'].dt.year
df2['month'] = df2['dtCreatedAt'].dt.month
df2['day'] = df2['dtCreatedAt'].dt.day
df2['dayofweek'] = df2['dtCreatedAt'].dt.dayofweek
df2['hour'] = df2['dtCreatedAt'].dt.hour
df2['quarter'] = df2['dtCreatedAt'].dt.quarter

df2["descMapName"].value_counts()

count
descMapName
de_mirage    52293
de_inferno   37875
de_dust2     21391
de_vertigo   17956
de_overpass  12738
de_nuke      12641
de_train     7033
de_ancient   3810

dtype: int64

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

df2['descMapName'] = le.fit_transform(df2['descMapName'])
df2["descMapName"].value_counts()

Показать скрытые выходные данные

[27] df2.info()

Показать скрытые выходные данные

[26] df2=df2.drop(["dtCreatedAt"], axis=1)
```

Рисунок 4 – Кодирование данных

Дальше построил матрицу корреляции данных (Рисунок 5).

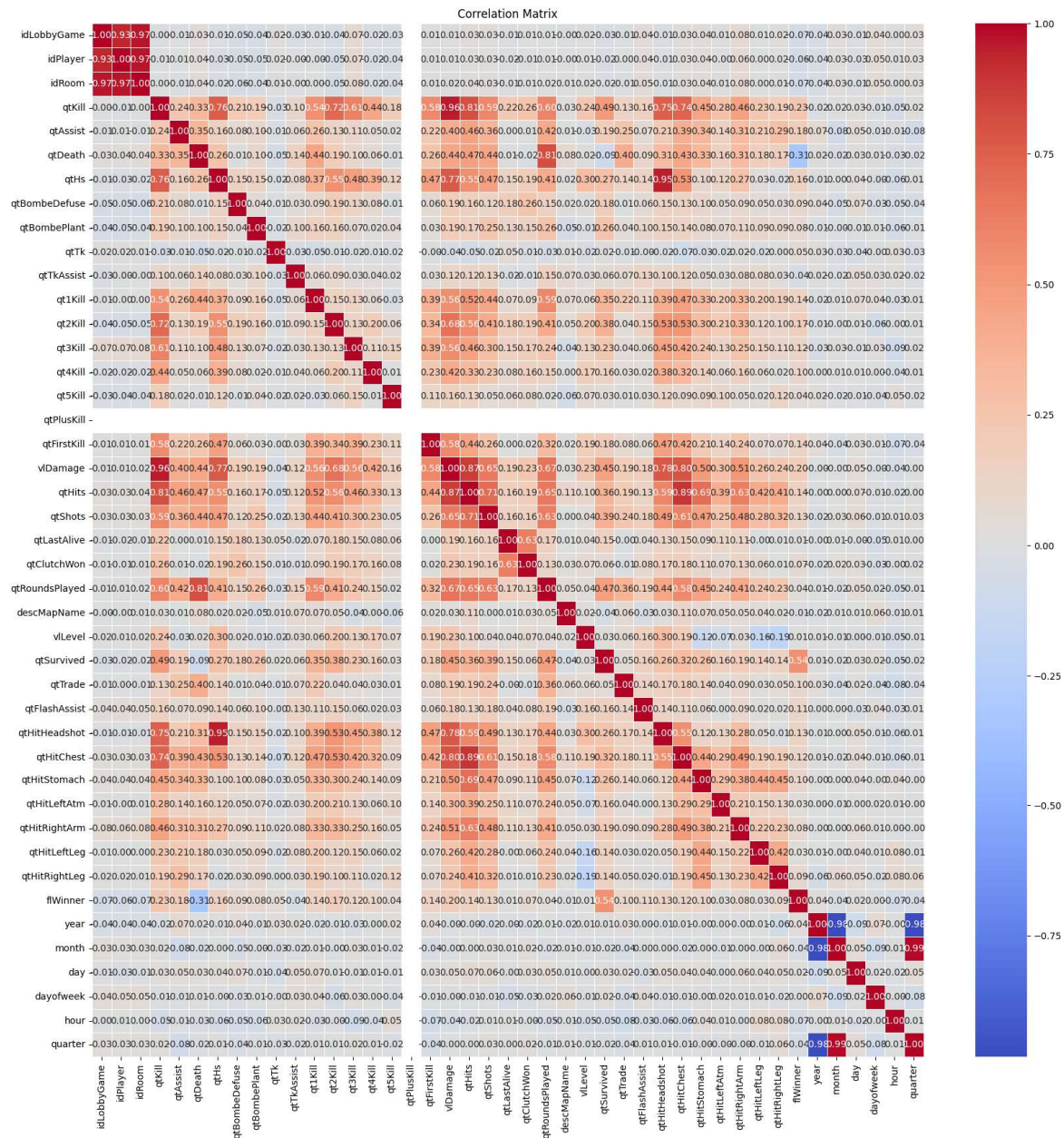


Рисунок 5 – Матрица корреляции данных

По данной матрице были определены параметры, наименее влияющие на другие, у которых коэффициент близок к 0. Удалил эти параметры и заполнил пропуски медианным значением (Рисунок 6).

```
df3 = df3.fillna(df3.median())

[49] df3 = df3.drop(['qtTk', 'day', 'year', 'month', 'qtPlusKill'], axis=1)
```

Рисунок 6 – Заполнение пропусков в столбцах

Построил ящики с усами для определения выбросов в данных (Рисунок 7).

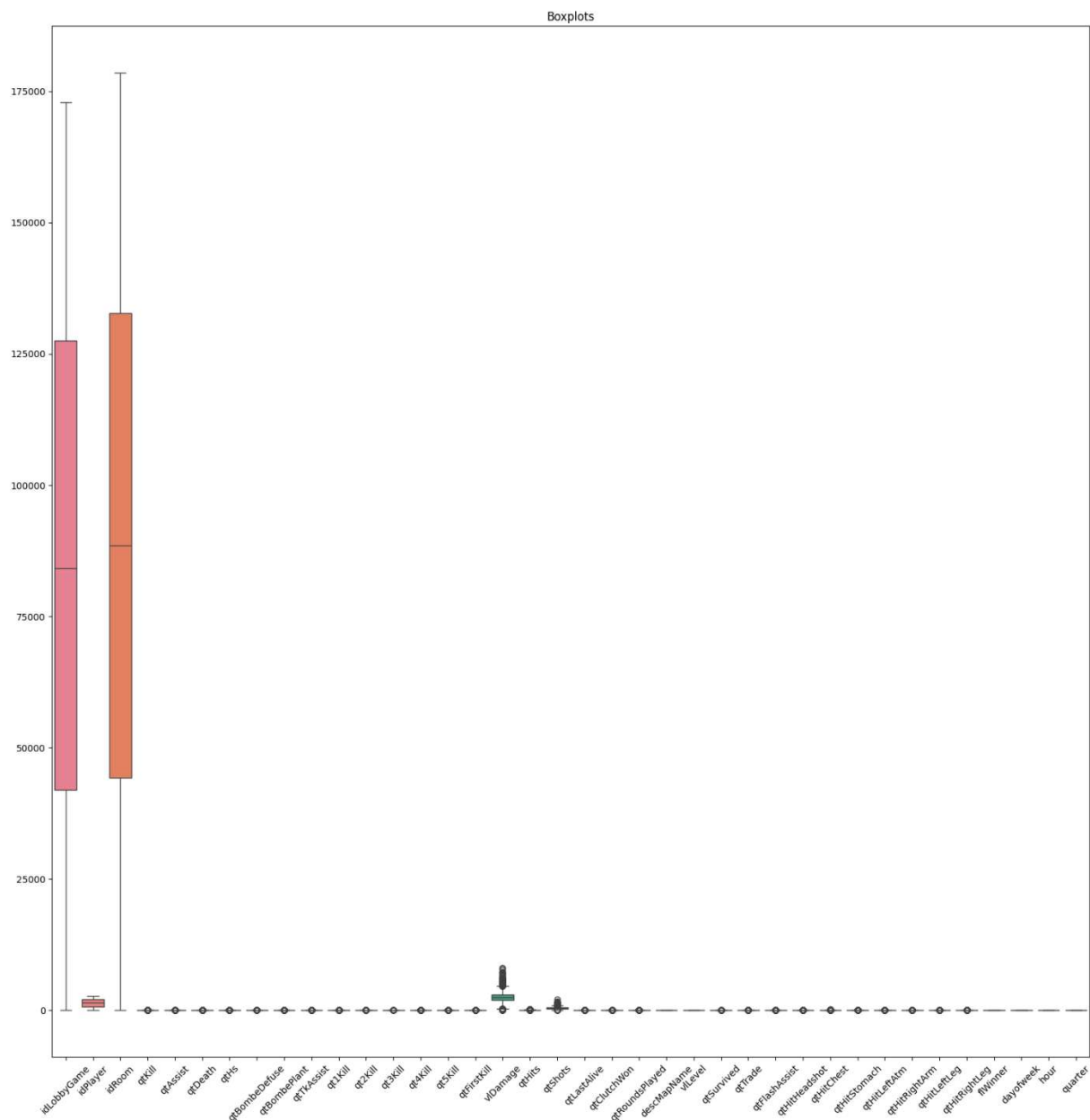


Рисунок 7 – Boxplot для каждого параметра

По графикам видно, что присутствуют статистические выбросы для двух признаков, однако, учитывая контекст данных, они могут иметь сильное влияние на результат (qFirstKill по крайней мере), потому было принято решение убрать только конкретно видимые выбросы vldamage (Рисунок 8).

```
[57] df3 = df3[df3['vldamage'] <= 6500]
```

Рисунок 8 – Чистка сильных выбросов

Дальше выполнил нормализацию признаков и подготовил данные для обучения (Рисунок 9).

```
[58] from sklearn.preprocessing import MinMaxScaler
      scaler = MinMaxScaler()
      df3 = pd.DataFrame(scaler.fit_transform(df3), columns=df3.columns)

[59] x = df3.drop(['flWinner'], axis=1)
      y = df3['flWinner']
```

Рисунок 9 – Завершение предобработки данных

Так как количество данных велико, обучение можно провести на меньшей выборке, с возможностью увеличить её в случае плохих результатов. Выборка представлена на Рисунке 10.

```
[60] x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)
```

Рисунок 10 – Выборка данных для обучения

Обучил модель линейной регрессии (Рисунок 13) и получил необходимые метрики (Рисунок 14).

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

model = LinearRegression()

model.fit(x_train, y_train)
```

LinearRegression ⓘ ?

LinearRegression()

Рисунок 13 – Модель линейной регрессии

```
[35] y_pred = model.predict(x_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Среднеквадратическая ошибка (MSE): {mse}")
print(f"Коэффициент детерминации R^2: {r2}")
```

↗ Среднеквадратическая ошибка (MSE): 0.00032002295956539523
Коэффициент детерминации R^2: 0.9500545141342607

Рисунок 14 – Метрики линейной регрессии

Был построен график соотношения предсказанных значений и реальных (Рисунок 15), а также график с понижением размерности с 40 до 1 (Рисунок 16).

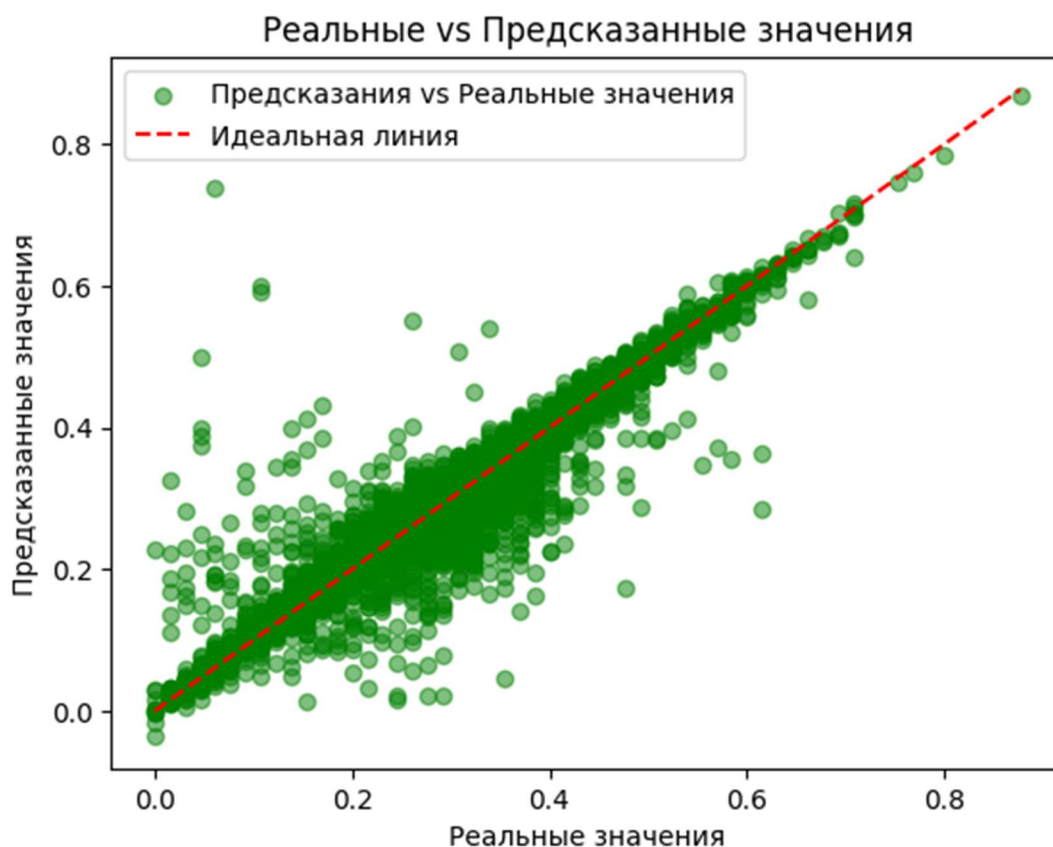


Рисунок 15 – График соотношения значений

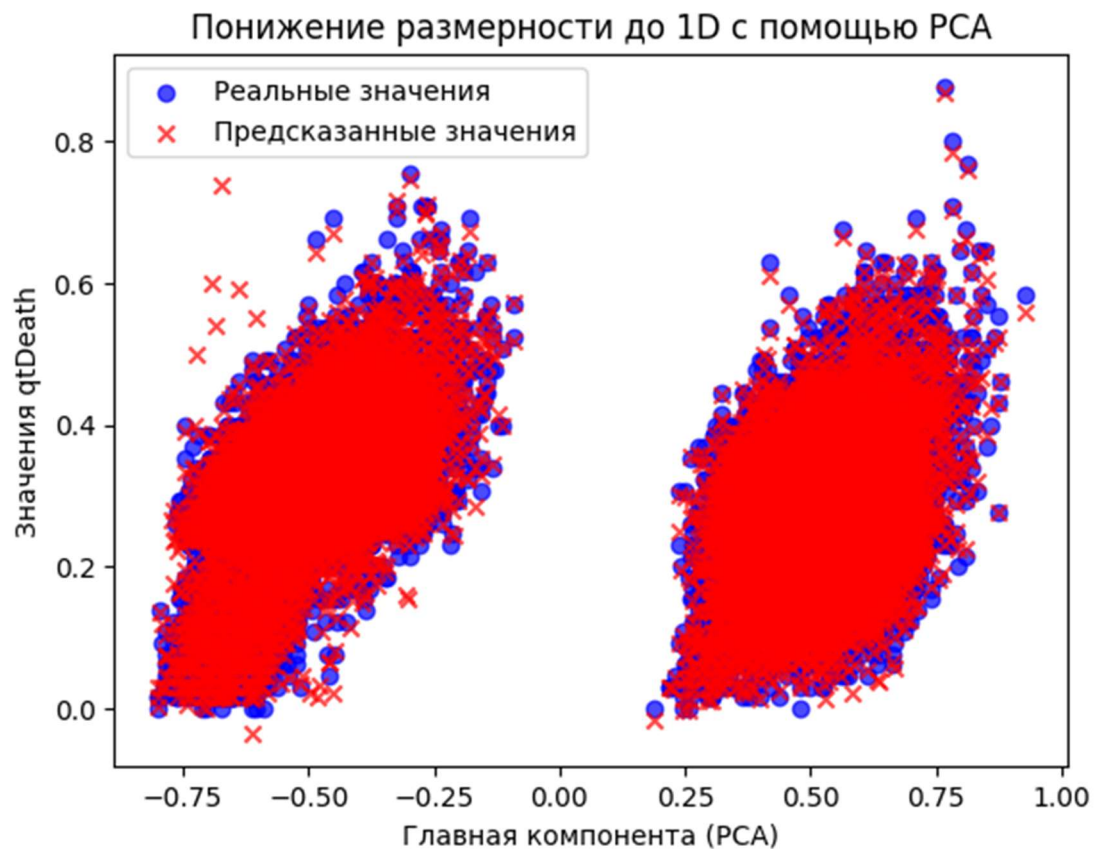


Рисунок 16 – График с понижением размерности

Обучил модель LASSO и получил необходимые метрики (Рисунок 17). После настройки метрики удалось улучшить.

```
lasso = Lasso(alpha=0.1)
lasso.fit(x_train, y_train)

y_pred = lasso.predict(x_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Среднеквадратическая ошибка (MSE): {mse}")
print(f"Коэффициент детерминации R^2: {r2}")
```

```
[>] Среднеквадратическая ошибка (MSE): 0.0064081499440193914
Коэффициент детерминации R^2: -0.00011000113620407426
```

```

lasso = Lasso(alpha=0.0000003)
lasso.fit(x_train, y_train)

y_pred = lasso.predict(x_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Среднеквадратическая ошибка (MSE): {mse}")
print(f"Коэффициент детерминации R^2: {r2}")

```

 Среднеквадратическая ошибка (MSE): 0.0003199681083983853
 Коэффициент детерминации R^2: 0.950063074670637

Рисунок 17 – Модель и метрики LASSO

Был построен график соотношения предсказанных значений и реальных (Рисунок 18), а также график с понижением размерности с 40 до 1 (Рисунок 19).

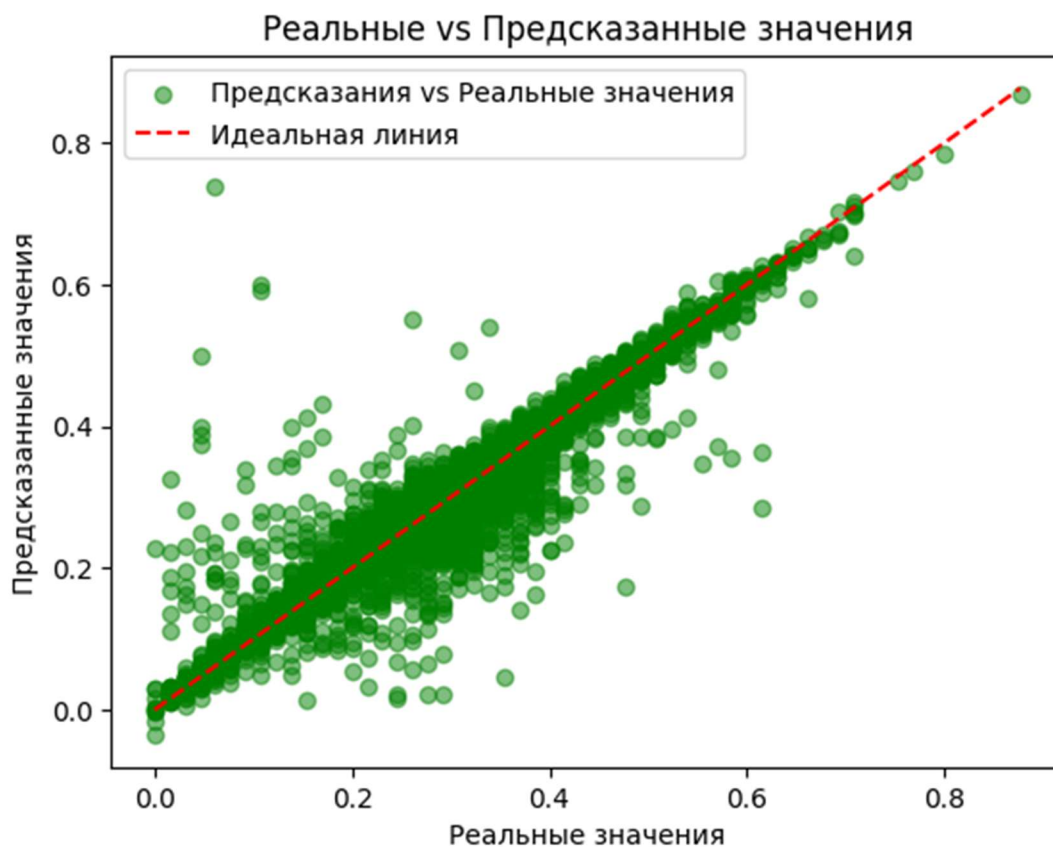


Рисунок 18 – График соотношения значений

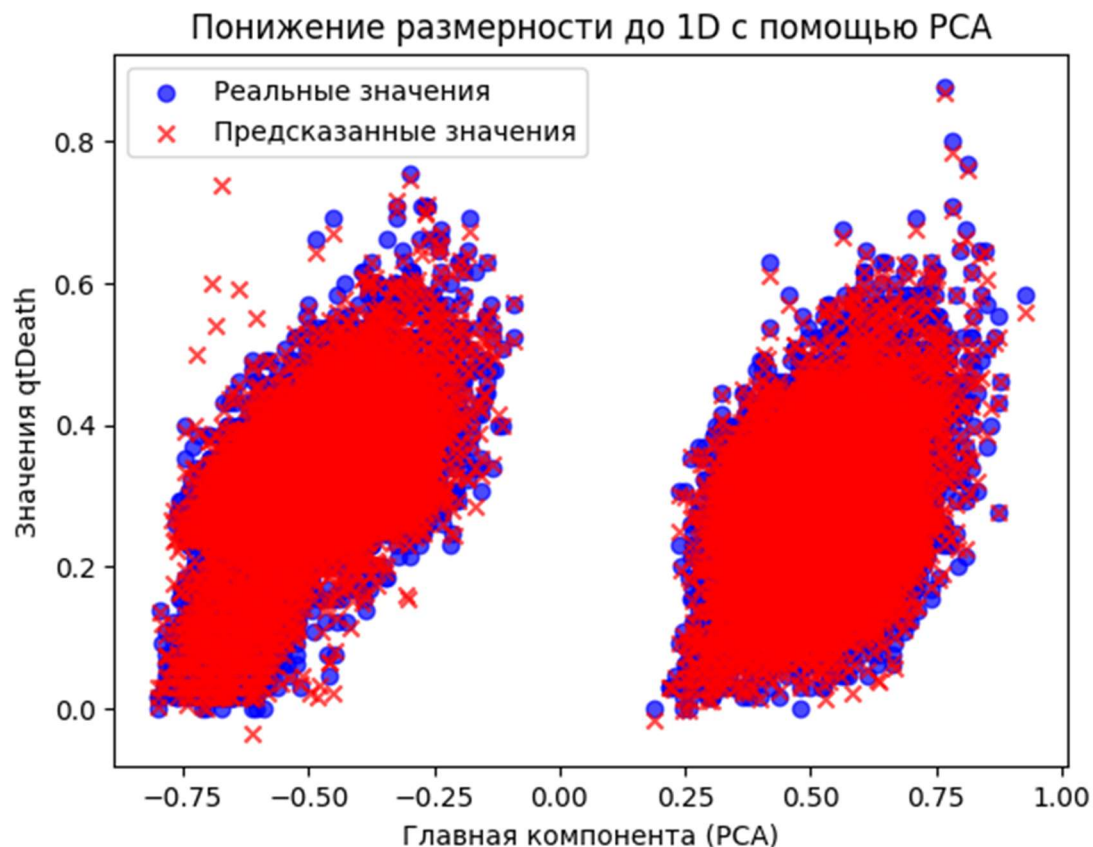


Рисунок 19 – График с понижением размерности

Обучил модель ридж-регрессии и получил необходимые метрики (Рисунок 20). После настройки метрики удалось улучшить.

```
ridge = Ridge(alpha=1.0)
ridge.fit(x_train, y_train)

y_pred = ridge.predict(x_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Среднеквадратическая ошибка (MSE): {mse}")
print(f"Коэффициент детерминации R^2: {r2}")
```



```
Среднеквадратическая ошибка (MSE): 0.00031980769366715363
Коэффициент детерминации R^2: 0.9500881103483967
```

Рисунок 20 – Модель и метрики ридж-регрессии

Был построен график соотношения предсказанных значений и реальных (Рисунок 21), а также график с понижением размерности с 40 до 1 (Рисунок 22).

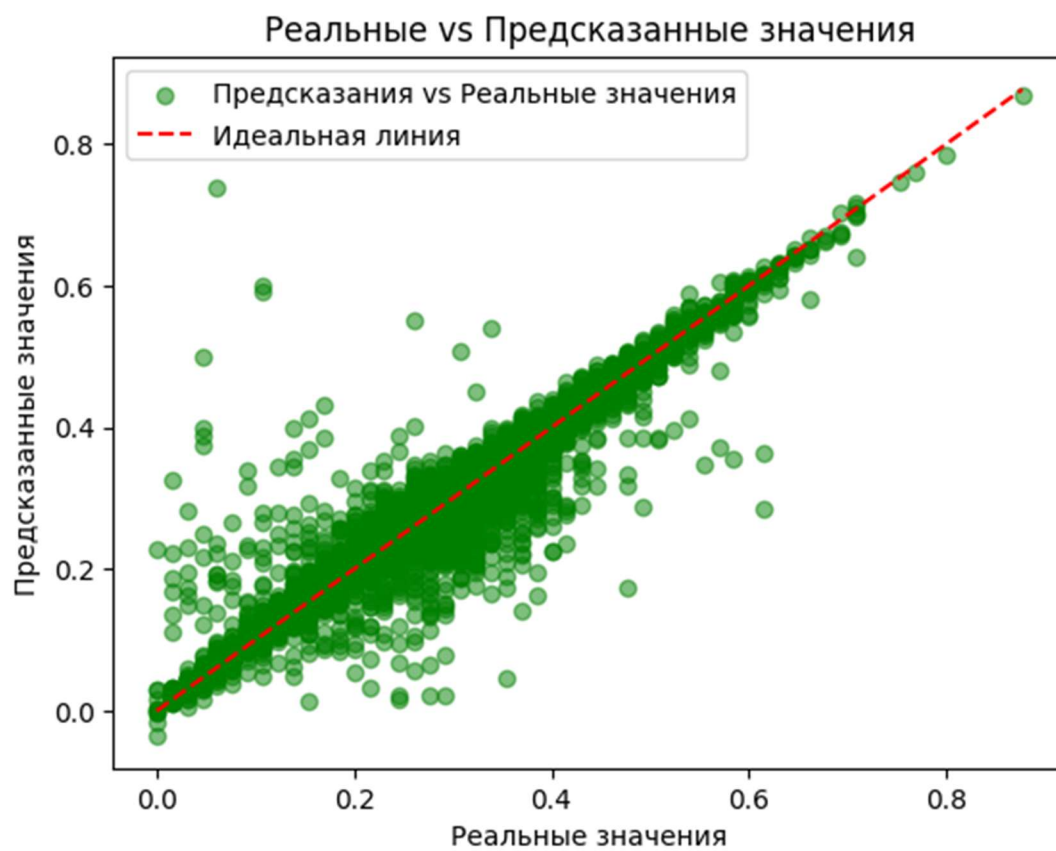


Рисунок 21 – График соотношения значений

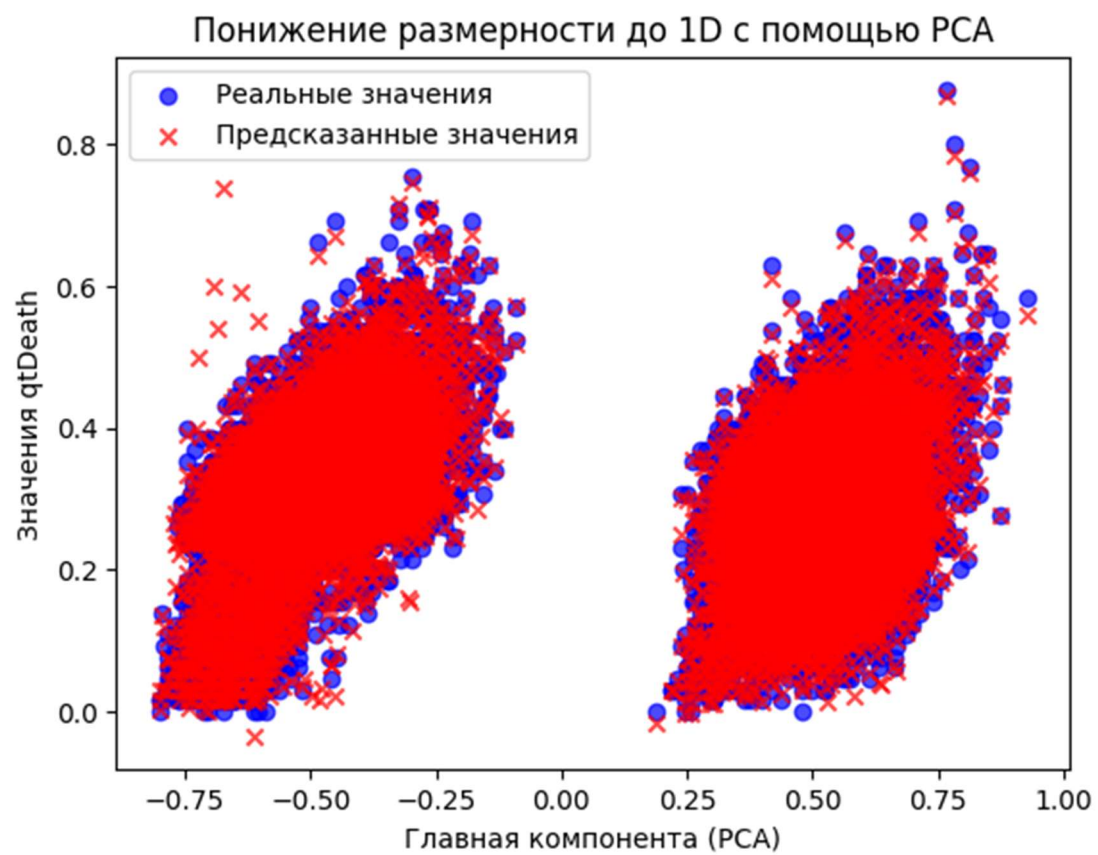


Рисунок 22 – График с понижением размерности

Выводы

В рамках данной лабораторной работы была проведена предварительная обработка данных аналогичная обработке в предыдущей работе. Этот процесс включал визуализацию значимых признаков, очистку данных от пропусков, дубликатов и аномалий, а также нормализацию, что способствовало повышению качества обучения моделей. Анализ корреляций позволил выявить избыточные признаки и избежать проблемы мультиколлинеарности.

На этапе построения моделей были использованы три популярных метода: линейная регрессия, LASSO и ридж-регрессия. Каждый из этих алгоритмов имеет свои особенности.

Оценка качества моделей проводилась с использованием метрик СКО и коэффициент детерминации, что позволило оценить их точность и полноту в решении задачи регрессии.

В целом качество получилось не плохим. Результаты всех 3 моделей мало отличаются друг от друга, что вероятно связано со спецификой данных и настройкой регуляризации.

Для улучшения метрик всех моделей, можно увеличить количество данных, провести тонкую настройку параметров для каждой из моделей. Однако это требует много времени и вычислительных ресурсов.

Приложения

Ссылка на блокнот в google.colab:

https://colab.research.google.com/drive/1iZsQ_JV_L63mawSvFnY9jKEHBJLz5tPv#scrollTo=2Z0RvMuiJcJX